```
import pandas as pd # For data loading, manipulation, and analysis of tabular d
import numpy as np # For numerical operations, especially with arrays.

# Text Preprocessing
import nltk # Natural Language Toolkit for tasks like tokenization, stemming, l
import re # Regular expression operations for advanced string matching and mani
import string # Provides constants and functions for common string operations,

# Feature Extraction (Vectorization)
from sklearn.feature_extraction.text import TfidfVectorizer # Converts a collec
from sklearn.feature_extraction.text import CountVectorizer # Converts a collec

# Model Building
from sklearn.model_selection import train_test_split # Splits arrays or matrice
from sklearn.naive_bayes import MultinomialNB # A probabilistic classifier ofte
from sklearn.linear_model import LogisticRegression # A linear model for binary
from sklearn.svm import SVC # Support Vector Classifier, a powerful and versati

# Model Evaluation
from sklearn.metrics import accuracy_score, classification_report, confusion_ma
import matplotlib.pyplot as plt # For creating static, interactive, and animate
import seaborn as sns # Built on Matplotlib, provides a high-level interface fo
```

```
# Load the news.csv file into a pandas DataFrame
df = pd.read_csv('/content/news.csv')

# Display the first 5 rows and the column information to understand the data st
print('First 5 rows of the dataset:')
display(df.head())

print('\nDataset Info:')
df.info()
```

```
First 5 rows of the dataset:
```

	Unnamed: 0	title	text	label
0	8476	You Can Smell Hillary's Fear	Daniel Greenfield, a Shillman Journalism Fello...	FAKE
1	10294	Watch The Exact Moment Paul Ryan Committed Pol...	Google Pinterest Digg Linkedin Reddit Stumbleu...	FAKE
2	3608	Kerry to go to Paris in gesture of sympathy	U.S. Secretary of State John F. Kerry said Mon...	REAL
3	10142	Bernie supporters on Twitter erupt in anger ag...	— Kaydee King (@KaydeeKing) November 9, 2016 T...	FAKE
4	875	The Battle of New York: Why This Primary Matters	It's primary day in New York and front-runners...	REAL

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6335 entries, 0 to 6334
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Unnamed: 0  6335 non-null   int64
 1   title       6335 non-null   object
 2   text        6335 non-null   object
 3   label       6335 non-null   object
dtypes: int64(1), object(3)
memory usage: 198.1+ KB
```

```
from nltk.corpus import stopwords

# 1. Download and load ONCE
nltk.download('stopwords')
stop_words = set(stopwords.words('english'))

def preprocess_text(text):
    """Performs text preprocessing steps including lowercase conversion, punctu

    Args:
        text (str): The input text to be preprocessed.

    Returns:
        str: The preprocessed text.
    """
    # Convert text to lowercase
    text = text.lower()
    # Remove punctuation using translate for efficiency
    text = text.translate(str.maketrans('', '', string.punctuation))

    # Tokenize the text into words
    words = text.split()
    # Remove stopwords by checking against a pre-loaded set
    words = [word for word in words if word not in stop_words]

    # Join the processed words back into a single string
```

```
    return ' '.join(words)

# Now apply it
X_train_processed = X_train.apply(preprocess_text)
X_test_processed = X_test.apply(preprocess_text)
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

```
# Initialize TF-IDF Vectorizer
tfidf_vectorizer = TfidfVectorizer(max_features=5000) # Limiting to 5000 featur

# Fit the vectorizer on the training data and transform both training and testi
X_train_tfidf = tfidf_vectorizer.fit_transform(X_train_processed)
X_test_tfidf = tfidf_vectorizer.transform(X_test_processed)

# Print feature matrix shape
print(f"Shape of TF-IDF feature matrix for training data: {X_train_tfidf.shape}
print(f"Shape of TF-IDF feature matrix for testing data: {X_test_tfidf.shape}")

# Display sample feature names (top 10)
print('\nSample TF-IDF Feature Names:')
display(tfidf_vectorizer.get_feature_names_out()[:10])
```

```
Shape of TF-IDF feature matrix for training data: (5068, 5000)
Shape of TF-IDF feature matrix for testing data: (1267, 5000)

Sample TF-IDF Feature Names:
array(['10', '100', '1000', '10000', '100000', '11', '12', '13', '14',
       '15'], dtype=object)
```

```
# Assuming 'text' is the feature and 'label' is the target variable
X = df['text']
y = df['label']

# Split the data into training and testing sets with 80% for training and 20% f
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

print(f"Shape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")
```

```
Shape of X_train: (5068,)
Shape of X_test: (1267,)
Shape of y_train: (5068,)
Shape of y_test: (1267,)
```

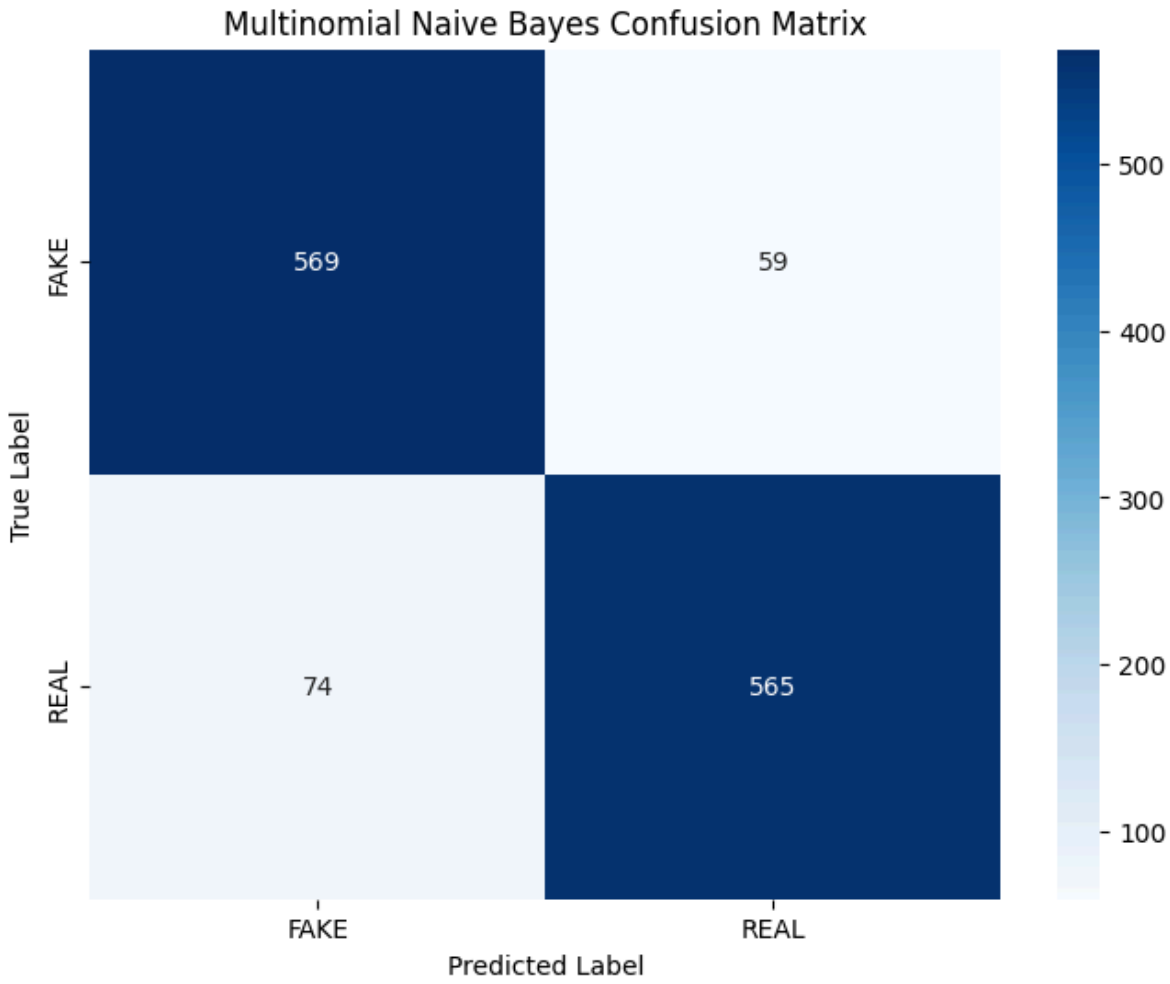
```
# Initialize the Multinomial Naive Bayes classifier
multinomial_nb_model = MultinomialNB()

# Train the model using the TF-IDF transformed training data
multinomial_nb_model.fit(X_train_tfidf, y_train)

# Display model parameters
```

```
print("Multinomial Naive Bayes Model Parameters:")
display(multinomial_nb_model.get_params())
```

```
Multinomial Naive Bayes Model Parameters:
{'alpha': 1.0, 'class_prior': None, 'fit_prior': True, 'force_alpha': True}
```

```
# Predict labels for the test set
y_pred_nb = multinomial_nb_model.predict(X_test_tfidf)

# Evaluate Accuracy
accuracy_nb = accuracy_score(y_test, y_pred_nb)
print(f"Multinomial Naive Bayes Model Accuracy: {accuracy_nb:.4f}\n")

# Display Classification Report (Precision, Recall, F1-score)
print("Multinomial Naive Bayes Classification Report:")
print(classification_report(y_test, y_pred_nb))

# Display Confusion Matrix
cm_nb = confusion_matrix(y_test, y_pred_nb)
plt.figure(figsize=(8, 6))
sns.heatmap(cm_nb, annot=True, fmt='d', cmap='Blues',
            xticklabels=multinomial_nb_model.classes_,
            yticklabels=multinomial_nb_model.classes_)
plt.title('Multinomial Naive Bayes Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

```
Multinomial Naive Bayes Model Accuracy: 0.8950

Multinomial Naive Bayes Classification Report:
              precision    recall  f1-score   support

        FAKE       0.88      0.91      0.90       628
        REAL       0.91      0.88      0.89       639

    accuracy                           0.90      1267
   macro avg       0.90      0.90      0.90      1267
weighted avg       0.90      0.90      0.90      1267
```

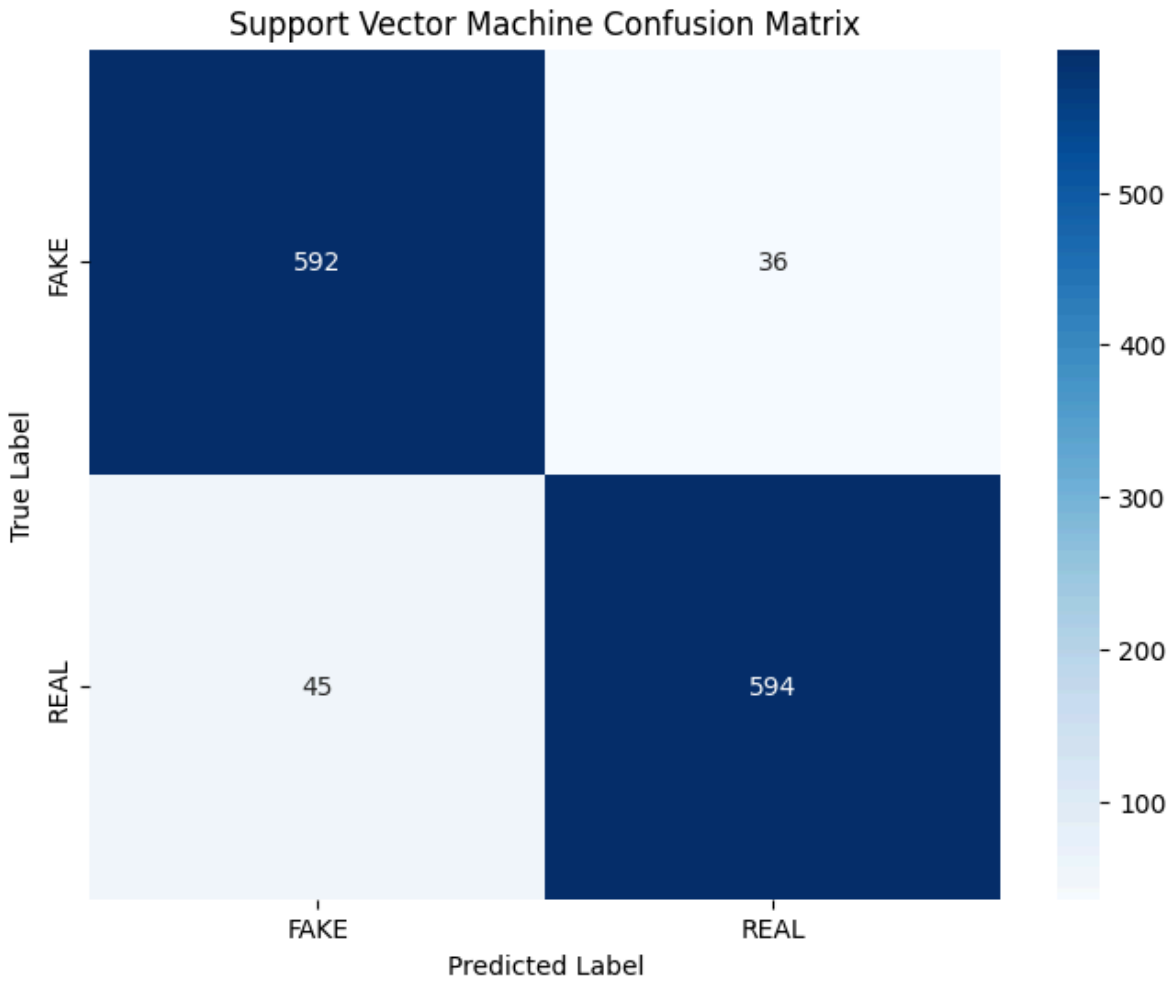



```
svc_model = SVC(random_state=42)

# Train the SVC model
svc_model.fit(X_train_tfidf, y_train)

# Display model parameters
print("Support Vector Machine Model Parameters:")
display(svc_model.get_params())
```

```
Support Vector Machine Model Parameters:
{'C': 1.0,
 'break_ties': False,
 'cache_size': 200,
 'class_weight': None,
 'coef0': 0.0,
 'decision_function_shape': 'ovr',
 'degree': 3,
 'gamma': 'scale',
 'kernel': 'rbf',
 'max_iter': -1,
 'probability': False,
 'random_state': 42,
 'shrinking': True,
 'tol': 0.001,
 'verbose': False}
```

```
y_pred_svc = svc_model.predict(X_test_tfidf)

accuracy_svc = accuracy_score(y_test, y_pred_svc)
print(f"Support Vector Machine Model Accuracy: {accuracy_svc:.4f}\n")

print("Support Vector Machine Classification Report:")
print(classification_report(y_test, y_pred_svc))

cm_svc = confusion_matrix(y_test, y_pred_svc)
plt.figure(figsize=(8, 6))
sns.heatmap(cm_svc, annot=True, fmt='d', cmap='Blues',
            xticklabels=svc_model.classes_,
            yticklabels=svc_model.classes_)
plt.title('Support Vector Machine Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

```
Support Vector Machine Model Accuracy: 0.9361

Support Vector Machine Classification Report:
              precision    recall  f1-score   support

        FAKE       0.93      0.94      0.94       628
        REAL       0.94      0.93      0.94       639

    accuracy                           0.94      1267
   macro avg       0.94      0.94      0.94      1267
weighted avg       0.94      0.94      0.94      1267
```




```
from sklearn.model_selection import GridSearchCV
print("GridSearchCV imported successfully.")
```

```
GridSearchCV imported successfully.
```

```
from sklearn.svm import LinearSVC
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import Pipeline
from sklearn.model_selection import RandomizedSearchCV

# 1. Faster Vectorizer (limit features to remove noise)
tfidf = TfidfVectorizer(stop_words='english', ngram_range=(1, 2), max_features=1

# 2. LinearSVC is much faster than SVC(kernel='linear')
lsvc = LinearSVC(dual=False, random_state=42, max_iter=2000)
```

```
# 3. Create a Pipeline to keep things clean
pipeline = Pipeline([
    ('tfidf', tfidf),
    ('clf', lsvc)
])

# 4. We only need to search 'C'. Higher C usually helps accuracy but watch for ov
param_dist = {
    'clf__C': [0.1, 1, 10, 50]
}

# n_iter=4 means only 4 combinations * 5 folds = 20 fits total. Much faster!
random_search = RandomizedSearchCV(pipeline, param_dist, n_iter=4, cv=5, verbose=

print("Starting Fast Search...")
random_search.fit(X_train, y_train) # Pass raw text, pipeline handles TF-IDF!

print(f"Best Score: {random_search.best_score_:.2%}")
print(f"Best Params: {random_search.best_params_}")
```

```
Starting Fast Search...
Fitting 5 folds for each of 4 candidates, totalling 20 fits
Best Score: 93.75%
Best Params: {'clf__C': 1}
```

```
# Install XGBoost library if not already installed
!pip install xgboost
print("XGBoost installed successfully.")
```

```
Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.12/dis
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages
XGBoost installed successfully.
```

Train XGBoost Model

Subtask:

Initialize and train an XGBoost Classifier model using the TF-IDF transformed training data. XGBoost is a powerful and efficient implementation of gradient boosting designed for speed and performance.

```
import xgboost as xgb
from sklearn.preprocessing import LabelEncoder

# Initialize LabelEncoder
le = LabelEncoder()

# Fit LabelEncoder on the unique labels from the training set and transform both
y_train_encoded = le.fit_transform(y_train)
y_test_encoded = le.transform(y_test)
```

```
# Store original class names for later use (e.g., plotting)
original_classes = le.classes_

# Initialize the XGBoost classifier
# For binary classification, use 'binary:logistic' objective
# eval_metric can be 'logloss' or 'error' (binary classification error rate)
xgboost_model = xgb.XGBClassifier(objective='binary:logistic', eval_metric='logl

# Train the model using the TF-IDF transformed training data and encoded labels
xgboost_model.fit(X_train_tfidf, y_train_encoded)

# Display model parameters
print("XGBoost Model Parameters:")
display(xgboost_model.get_params())
```

```
XGBoost Model Parameters:
{'objective': 'binary:logistic',
 'base_score': None,
 'booster': None,
 'callbacks': None,
 'colsample_bylevel': None,
 'colsample_bynode': None,
 'colsample_bytree': None,
 'device': None,
 'early_stopping_rounds': None,
 'enable_categorical': False,
 'eval_metric': 'logloss',
 'feature_types': None,
 'feature_weights': None,
 'gamma': None,
 'grow_policy': None,
 'importance_type': None,
 'interaction_constraints': None,
 'learning_rate': None,
 'max_bin': None,
 'max_cat_threshold': None,
 'max_cat_to_onehot': None,
 'max_delta_step': None,
 'max_depth': None,
 'max_leaves': None,
 'min_child_weight': None,
 'missing': nan,
 'monotone_constraints': None,
 'multi_strategy': None,
 'n_estimators': None,
 'n_jobs': None,
 'num_parallel_tree': None,
 'random_state': 42,
 'reg_alpha': None,
 'reg_lambda': None,
 'sampling_method': None,
 'scale_pos_weight': None,
 'subsample': None,
 'tree_method': None,
 'validate_parameters': None,
 'verbosity': None}
```

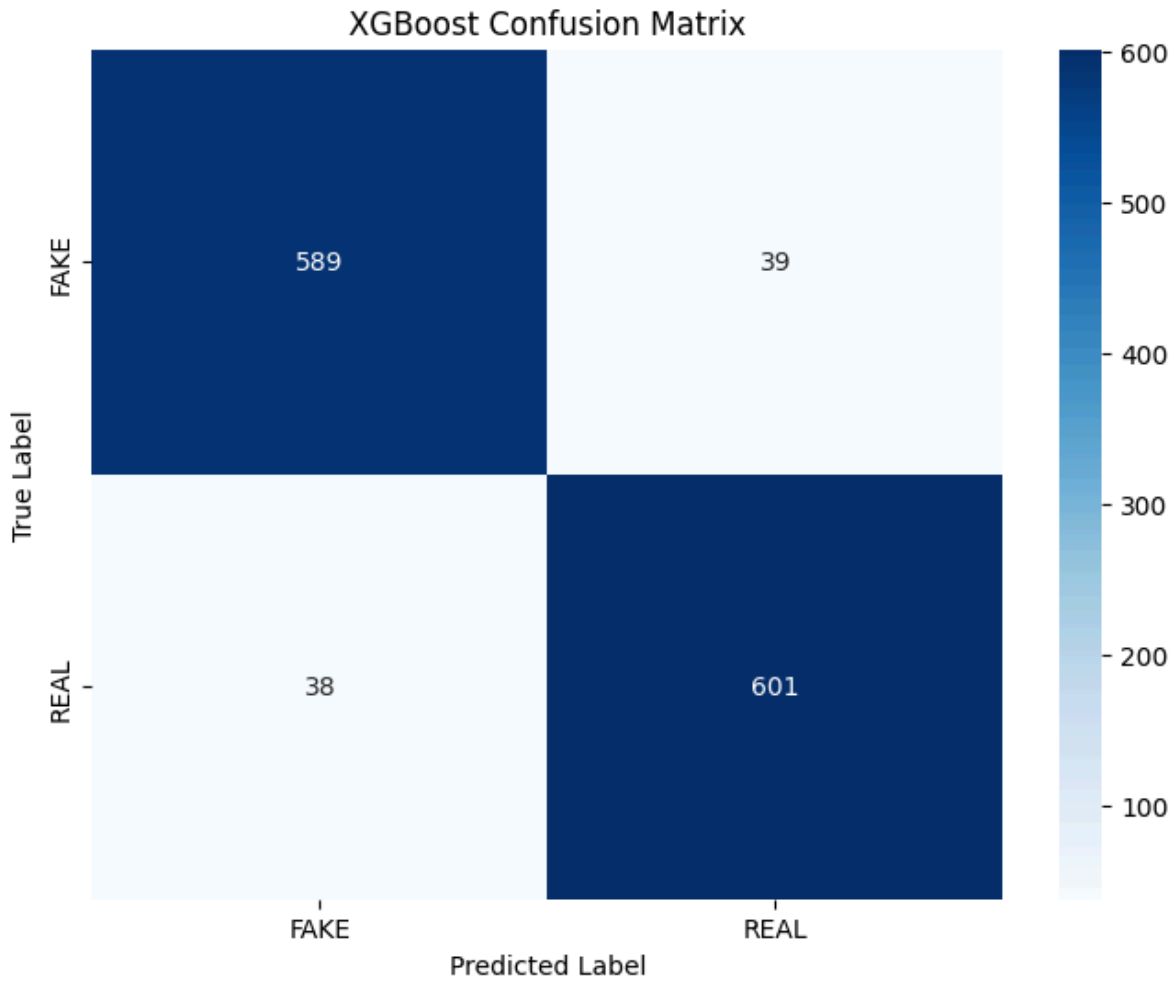
Evaluate XGBoost Model

Subtask:

Predict labels for the test set using the trained XGBoost model and evaluate its performance using accuracy, classification report, and confusion matrix.

```
y_pred_xgb = xgboost_model.predict(X_test_tfidf)

# Evaluate Accuracy using encoded true labels
accuracy_xgb = accuracy_score(y_test_encoded, y_pred_xgb)
print(f"XGBoost Model Accuracy: {accuracy_xgb:.4f}\n")

# Display Classification Report (Precision, Recall, F1-score) using encoded tru
print("XGBoost Classification Report:")
print(classification_report(y_test_encoded, y_pred_xgb, target_names=original_c

# Display Confusion Matrix using encoded true labels
cm_xgb = confusion_matrix(y_test_encoded, y_pred_xgb)
plt.figure(figsize=(8, 6))
sns.heatmap(cm_xgb, annot=True, fmt='d', cmap='Blues',
            xticklabels=original_classes,
            yticklabels=original_classes)
plt.title('XGBoost Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

```
XGBoost Model Accuracy: 0.9392

XGBoost Classification Report:
              precision    recall  f1-score   support

        FAKE       0.94      0.94      0.94       628
        REAL       0.94      0.94      0.94       639

    accuracy                           0.94      1267
   macro avg       0.94      0.94      0.94      1267
weighted avg       0.94      0.94      0.94      1267
```



Evaluate Best LinearSVC Model from Randomized Search

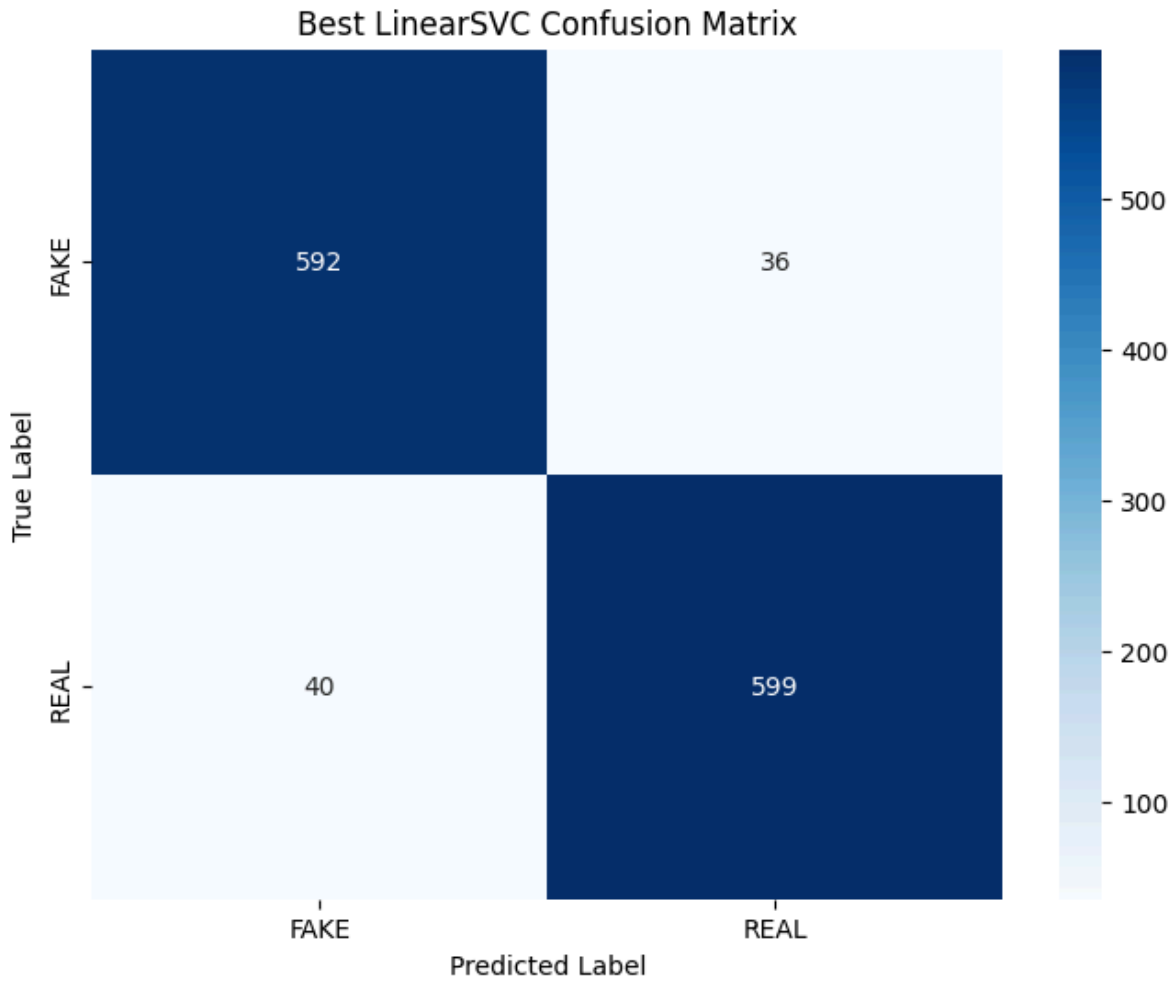
Subtask:

Predict labels for the test set using the best LinearSVC model found by `RandomizedSearchCV` and evaluate its performance using accuracy, classification report, and confusion matrix.

```
# Get the best estimator (the trained LinearSVC pipeline) from RandomizedSearch
best_linear_svc_model = random_search.best_estimator_

# Predict labels for the test set using the best model
```

```
y_pred_best_lsvc = best_linear_svc_model.predict(X_test)

# Evaluate Accuracy
accuracy_best_lsvc = accuracy_score(y_test, y_pred_best_lsvc)
print(f"Best LinearSVC Model Accuracy: {accuracy_best_lsvc:.4f}\n")

# Display Classification Report (Precision, Recall, F1-score)
print("Best LinearSVC Classification Report:")
print(classification_report(y_test, y_pred_best_lsvc))

# Display Confusion Matrix
cm_best_lsvc = confusion_matrix(y_test, y_pred_best_lsvc)
plt.figure(figsize=(8, 6))
sns.heatmap(cm_best_lsvc, annot=True, fmt='d', cmap='Blues',
            xticklabels=best_linear_svc_model.named_steps['clf'].classes_,
            yticklabels=best_linear_svc_model.named_steps['clf'].classes_)
plt.title('Best LinearSVC Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

```
Best LinearSVC Model Accuracy: 0.9400

Best LinearSVC Classification Report:
              precision    recall  f1-score   support

        FAKE       0.94      0.94      0.94       628
        REAL       0.94      0.94      0.94       639

    accuracy                           0.94      1267
   macro avg       0.94      0.94      0.94      1267
weighted avg       0.94      0.94      0.94      1267
```



Train Gradient Boosting Model

Subtask:

Initialize and train a Gradient Boosting Classifier model using the TF-IDF transformed training data. Gradient Boosting is an ensemble method that builds a strong predictive model from a sequence of weaker models.

```
from sklearn.ensemble import GradientBoostingClassifier

# Initialize the Gradient Boosting classifier
gradient_boosting_model = GradientBoostingClassifier(random_state=42)
```

```
# Train the model using the TF-IDF transformed training data
gradient_boosting_model.fit(X_train_tfidf, y_train)

# Display model parameters
print("Gradient Boosting Model Parameters:")
display(gradient_boosting_model.get_params())
```

```
Gradient Boosting Model Parameters:
{'ccp_alpha': 0.0,
 'criterion': 'friedman_mse',
 'init': None,
 'learning_rate': 0.1,
 'loss': 'log_loss',
 'max_depth': 3,
 'max_features': None,
 'max_leaf_nodes': None,
 'min_impurity_decrease': 0.0,
 'min_samples_leaf': 1,
 'min_samples_split': 2,
 'min_weight_fraction_leaf': 0.0,
 'n_estimators': 100,
 'n_iter_no_change': None,
 'random_state': 42,
 'subsample': 1.0,
 'tol': 0.0001,
 'validation_fraction': 0.1,
 'verbose': 0,
 'warm_start': False}
```

Evaluate Gradient Boosting Model

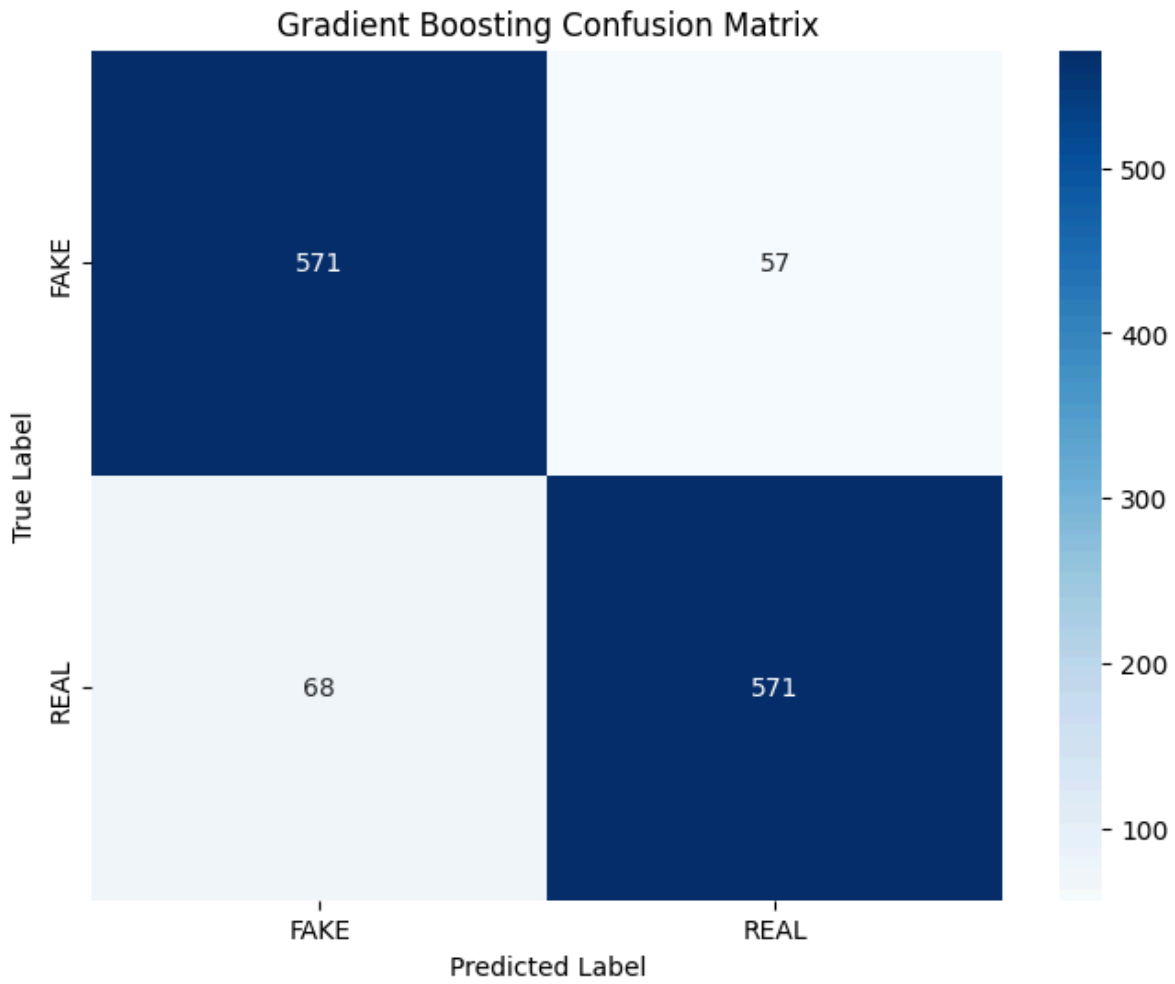
Subtask:

Predict labels for the test set using the trained Gradient Boosting model and evaluate its performance using accuracy, classification report, and confusion matrix.

```
y_pred_gb = gradient_boosting_model.predict(X_test_tfidf)

# Evaluate Accuracy
accuracy_gb = accuracy_score(y_test, y_pred_gb)
print(f"Gradient Boosting Model Accuracy: {accuracy_gb:.4f}\n")

# Display Classification Report (Precision, Recall, F1-score)
print("Gradient Boosting Classification Report:")
print(classification_report(y_test, y_pred_gb))

# Display Confusion Matrix
cm_gb = confusion_matrix(y_test, y_pred_gb)
plt.figure(figsize=(8, 6))
sns.heatmap(cm_gb, annot=True, fmt='d', cmap='Blues',
            xticklabels=gradient_boosting_model.classes_,
            yticklabels=gradient_boosting_model.classes_)
plt.title('Gradient Boosting Confusion Matrix')
plt.xlabel('Predicted Label')
```

```
plt.ylabel('True Label')
plt.show()
```

```
Gradient Boosting Model Accuracy: 0.9013

Gradient Boosting Classification Report:
              precision    recall  f1-score   support

        FAKE       0.89      0.91      0.90       628
        REAL       0.91      0.89      0.90       639

    accuracy                           0.90      1267
   macro avg       0.90      0.90      0.90      1267
weighted avg       0.90      0.90      0.90      1267
```



Train Decision Tree Model

Subtask:

Initialize and train a Decision Tree model using the TF-IDF transformed training data.

```
from sklearn.tree import DecisionTreeClassifier

# Initialize the Decision Tree classifier
decision_tree_model = DecisionTreeClassifier(random_state=42)
```

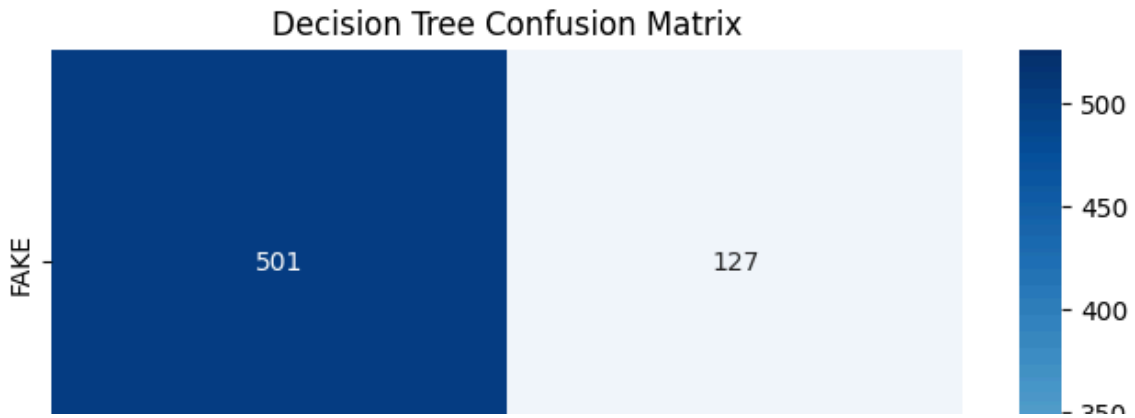
```
# Train the model using the TF-IDF transformed training data
decision_tree_model.fit(X_train_tfidf, y_train)

# Display model parameters
print("Decision Tree Model Parameters:")
display(decision_tree_model.get_params())
```

```
Decision Tree Model Parameters:
{'ccp_alpha': 0.0,
 'class_weight': None,
 'criterion': 'gini',
 'max_depth': None,
 'max_features': None,
 'max_leaf_nodes': None,
 'min_impurity_decrease': 0.0,
 'min_samples_leaf': 1,
 'min_samples_split': 2,
 'min_weight_fraction_leaf': 0.0,
 'monotonic_cst': None,
 'random_state': 42,
 'splitter': 'best'}
```

Evaluate Decision Tree Model

Subtask:

Predict labels for the test set using the trained Decision Tree model and evaluate its performance using accuracy, classification report, and confusion matrix.

```
y_pred_dt = decision_tree_model.predict(X_test_tfidf)

# Evaluate Accuracy
accuracy_dt = accuracy_score(y_test, y_pred_dt)
print(f"Decision Tree Model Accuracy: {accuracy_dt:.4f}\n")

# Display Classification Report (Precision, Recall, F1-score)
print("Decision Tree Classification Report:")
print(classification_report(y_test, y_pred_dt))

# Display Confusion Matrix
cm_dt = confusion_matrix(y_test, y_pred_dt)
plt.figure(figsize=(8, 6))
sns.heatmap(cm_dt, annot=True, fmt='d', cmap='Blues',
            xticklabels=decision_tree_model.classes_,
            yticklabels=decision_tree_model.classes_)
plt.title('Decision Tree Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

```
Decision Tree Model Accuracy: 0.8106

Decision Tree Classification Report:
              precision    recall  f1-score   support

        FAKE       0.82      0.80      0.81       628
        REAL       0.81      0.82      0.81       639

    accuracy                           0.81      1267
   macro avg       0.81      0.81      0.81      1267
weighted avg       0.81      0.81      0.81      1267
```